Université Abdelmalek Essaâdi
École Nationale des Sciences
Appliquées Tanger

Rapport
TD 2

Comparaison entre un algorithme exact et un algorithme heuristique : BFS contre A* sur une grille de cheminement

Réalisé par :
EL AMRAOUI Younes

Encadré par :
PR. SARA IBN EL AHRACHE

Module : Éthique de l'IA

Filière : GINF 2

Année universitaire : 2026/2027

Introduction

Ce TD compare deux manières de résoudre un problème de recherche de chemin sur une grille (un labyrinthe) : un algorithme **exact**, le parcours en largeur BFS (Breadth-First Search), et un algorithme **heuristique**, A* (A-star). L'objectif est de comparer la performance et la qualité des solutions selon trois critères : le nombre de nœuds explorés, le temps d'exécution et la longueur du chemin trouvé.

Pour cela, j'ai développé une application web interactive qui génère une grille, lance les deux algorithmes, anime leur exploration côte à côte et conserve l'historique des mesures. Le code source est disponible sur GitHub :

`https://github.com/youneselamraoui/BFS-AStar-Pathfinding-Visualizer`

Partie 1 : Implémentation

1. Architecture de l'application

- **Backend** (Python, Flask) : le dossier `src/pathfinding` contient `algorithms.py` (BFS et A*), `heuristics.py`, `grid.py` (génération de la grille) et `metrics.py` (mesures). L'API expose les routes `/api/grid` et `/api/run`. Des tests `pytest` valident le code.
- **Frontend** (HTML, Tailwind CSS, JavaScript, Chart.js) : paramétrage, visualisation sur deux canvas, tableau comparatif et graphiques.
- **Déploiement** : Docker et Docker Compose, accessible sur `localhost:5000`.

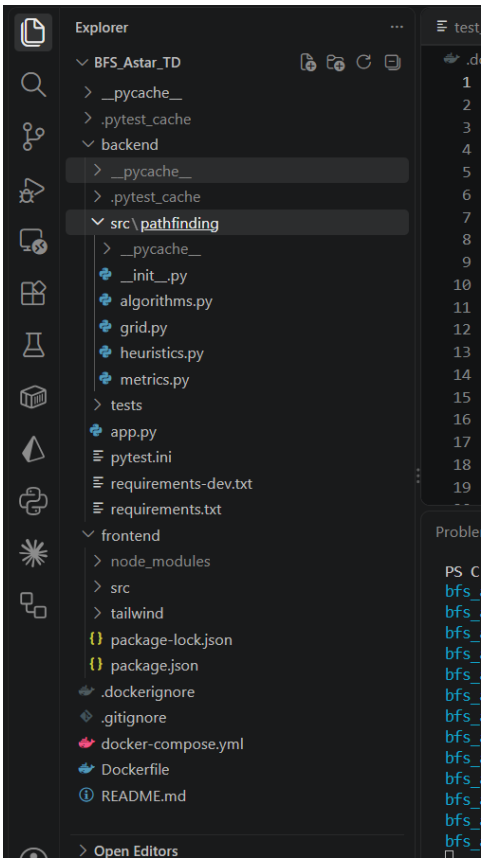


FIGURE 1 – Structure du projet.

2. Paramètres de l'expérience

La grille est générée avec une densité de murs réglable et une **seed** reproductible : deux exécutions avec la même seed donnent exactement la même grille, ce qui rend la comparaison équitable. Les mesures ci-dessous utilisent 20 % de murs et la seed 42. La taille varie de 20×20 à 100×100, le départ est en haut à gauche et l'arrivée en bas à droite. Les déplacements sont à coût unitaire.

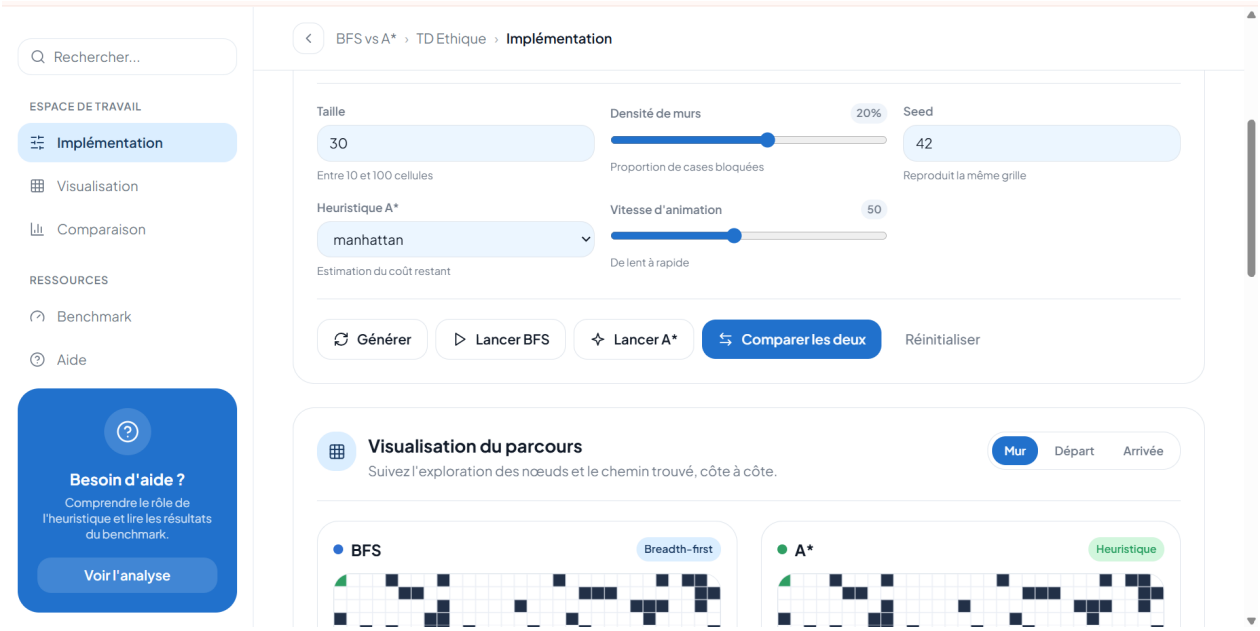


FIGURE 2 – Interface de paramétrage : taille, densité de murs, seed, heuristique et vitesse d'animation.

3. Les algorithmes

BFS explore les cases par couches successives à l'aide d'une file (FIFO), à partir du départ, jusqu'à atteindre l'arrivée. Sur un graphe à coûts unitaires, il garantit le plus court chemin. Sa complexité est $O(V + E)$, soit $O(N^2)$ pour une grille $N \times N$.

A* utilise une file de priorité ordonnée par

$$f(n) = g(n) + h(n),$$

où $g(n)$ est le coût réel du départ jusqu'à n et $h(n)$ une estimation du coût restant jusqu'à l'arrivée. Si h est **admissible** (elle ne surestime jamais le coût réel), A* renvoie un chemin optimal.

Quatre heuristiques sont disponibles dans l'application :

- **Manhattan** : $|x_1 - x_2| + |y_1 - y_2|$, adaptée aux déplacements à quatre directions.
- **Euclidienne** : distance à vol d'oiseau.
- **Chebyshev** : $\max(|x_1 - x_2|, |y_1 - y_2|)$, pensée pour les diagonales.
- **Zéro** : $h = 0$, ce qui ramène A* à l'algorithme de Dijkstra.

Concepts clés

Cette section résume les notions théoriques utilisées par l'application et sur lesquelles s'appuie l'analyse des résultats.

1. BFS : exploration en largeur, optimal sur coûts unitaires, file en $O(1)$

- **Exploration en largeur uniforme** : BFS développe d'abord tous les nœuds à distance 1 du départ, puis ceux à distance 2, et ainsi de suite. L'exploration progresse de manière régulière dans toutes les directions, sans aucune préférence pour le but, ce qui explique la large zone explorée visible sur nos captures.
- **Optimal sur coûts unitaires** : comme les nœuds sont visités par distance croissante, le premier chemin qui atteint l'arrivée est le plus court possible. Sur notre grille, où chaque déplacement coûte 1, BFS est donc toujours optimal. Cette garantie disparaît si les déplacements ont des coûts différents.
- **File en $O(1)$** : BFS utilise une simple file FIFO, dont l'ajout et le retrait d'un élément se font en temps constant. C'est moins coûteux que la file de priorité d'A* (tas binaire en $O(\log n)$), ce qui explique pourquoi BFS peut être plus rapide sur de petites grilles malgré un plus grand nombre de nœuds explorés.

2. A* : $f(n) = g(n) + h(n)$

A* choisit à chaque étape le nœud qui minimise $f(n) = g(n) + h(n)$, où $g(n)$ est le coût déjà parcouru et $h(n)$ l'estimation du coût restant. Lorsque l'heuristique est **admissible** (elle ne surestime jamais le coût réel), A* garantit un chemin optimal tout en explorant moins de nœuds que BFS, car il évite les zones qui éloignent de l'arrivée. Les résultats le confirment : 79 % à 87 % de nœuds en moins avec Manhattan.

3. Les heuristiques disponibles

- **Manhattan** (grille Manhattan) : $|x_1 - x_2| + |y_1 - y_2|$. C'est la distance exacte sans obstacles lorsque l'on ne peut se déplacer qu'en haut, bas, gauche, droite. Elle est admissible et très informative sur notre grille, d'où les meilleurs résultats.
- **Euclidienne** : $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$, distance à vol d'oiseau. Elle est admissible, mais plus faible que Manhattan sur une grille à quatre directions : elle sous-estime davantage le coût réel et guide donc un peu moins bien la recherche.
- **Chebyshev** (diagonales autorisées) : $\max(|x_1 - x_2|, |y_1 - y_2|)$. C'est la distance exacte lorsque les déplacements en diagonale coûtent 1. Dans notre application les déplace-

ments sont à quatre directions : elle sous-estime donc fortement le coût, reste admissible mais guide peu, ce qui explique les 7384 nœuds explorés (contre 999 avec Manhattan).

- **Zéro** (Dijkstra) : $h(n) = 0$, donc $f(n) = g(n)$. A* ne dispose d'aucune information sur la position du but et se comporte comme l'algorithme de Dijkstra. Avec des coûts unitaires, il explore autant de nœuds que BFS (694 sur 30×30, réduction de 0 %). Ce cas sert de référence pour mesurer l'apport réel d'une heuristique.

Règle générale : plus l'heuristique est proche du coût réel sans le dépasser, moins A* explore de nœuds. Une heuristique qui dépasse le coût réel perdrait la garantie d'optimalité.

4. Quand BFS reste préférable

- **Absence d'heuristique fiable** : sans bonne estimation du coût restant, A* n'a aucun avantage (cas $h = 0$) et ajoute le coût de sa file de priorité.
- **Petit graphe** : le nombre de nœuds est faible dans tous les cas, donc le gain d'A* est négligeable, voire négatif en temps.
- **Besoin de simplicité** : BFS ne demande qu'une file et une liste de nœuds visités, sans choix d'heuristique ni risque de la mal concevoir. Il est plus facile à écrire, à tester et à justifier.

Partie 2 : Résultats

1. Tableau comparatif des performances

Le tableau suivant est extrait de l'historique de l'application (heuristique Manhattan, 20 % de murs, seed 42). La réduction correspond à $1 - N_{\mathrm{A}^*} N_{\mathrm{BFS}}$, où N est le nombre de nœuds explorés par l'algorithme.

Grille	Nœuds BFS	Nœuds A*	Temps BFS (ms)	Temps A* (ms)	Longueur	Réduction
20×20	311	65	0,433	0,229	38	79,1 %
30×30	694	129	1,044	0,620	58	81,4 %
30×30 (bis)	694	129	2,387	0,429	58	81,4 %
50×50	1972	268	2,751	0,568	98	86,4 %
100×100	7944	999	11,832	2,750	198	87,4 %

TABLE 1 – Comparaison BFS / A* (Manhattan). La longueur du chemin est identique pour les deux algorithmes. La ligne « bis » est une seconde exécution de la même grille.

Cas particulier, avec l'heuristique nulle sur la grille 30×30 : A* explore **694 nœuds**, comme BFS (réduction de 0 %), avec un temps de 1,954 ms contre 0,830 ms pour BFS.

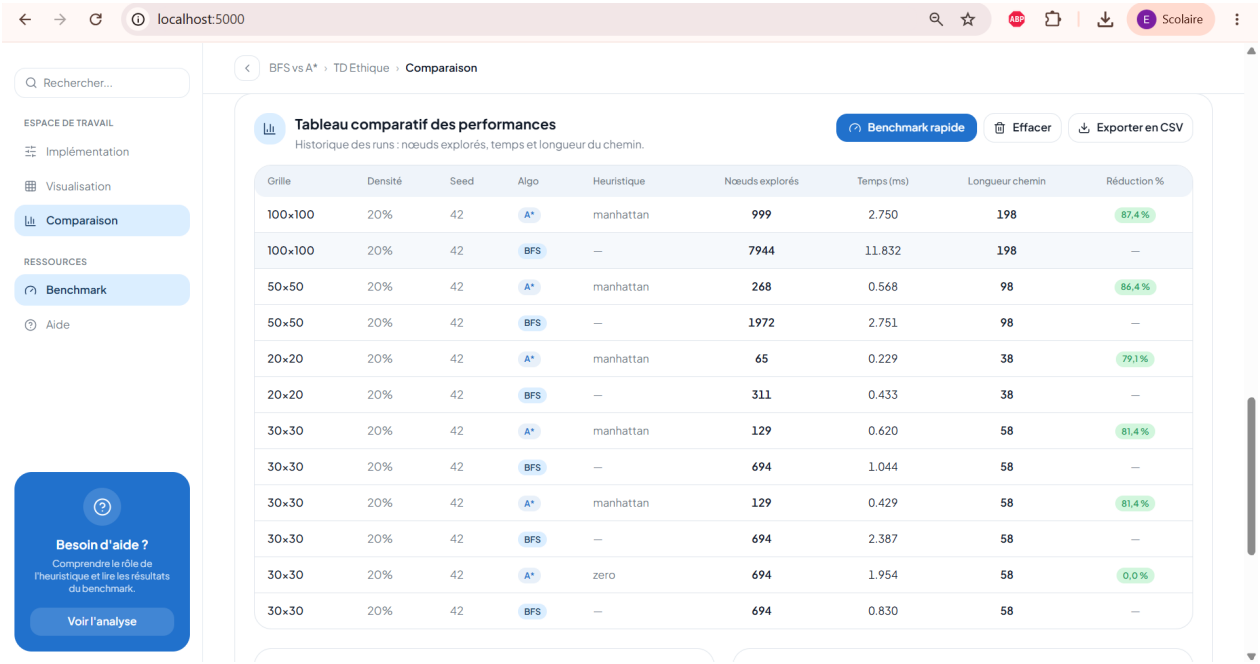


FIGURE 3 – Tableau comparatif généré par l'application.

2. Visualisation des parcours

Sur les captures, les cases bleu clair sont les nœuds explorés, la ligne bleue foncée est le chemin final et les cases noires sont les murs. Sur la grille 30×30, BFS explore une large zone autour du départ avant d'atteindre l'arrivée, alors que A* se concentre sur la diagonale vers l'arrivée. Les deux captures ont été prises pendant l'animation (état « en cours »), les compteurs de nœuds affichés sont donc partiels. Les valeurs finales sont celles du tableau.



FIGURE 4 – Exploration BFS (gauche) et A* (droite) sur une grille 30×30, seed 42.

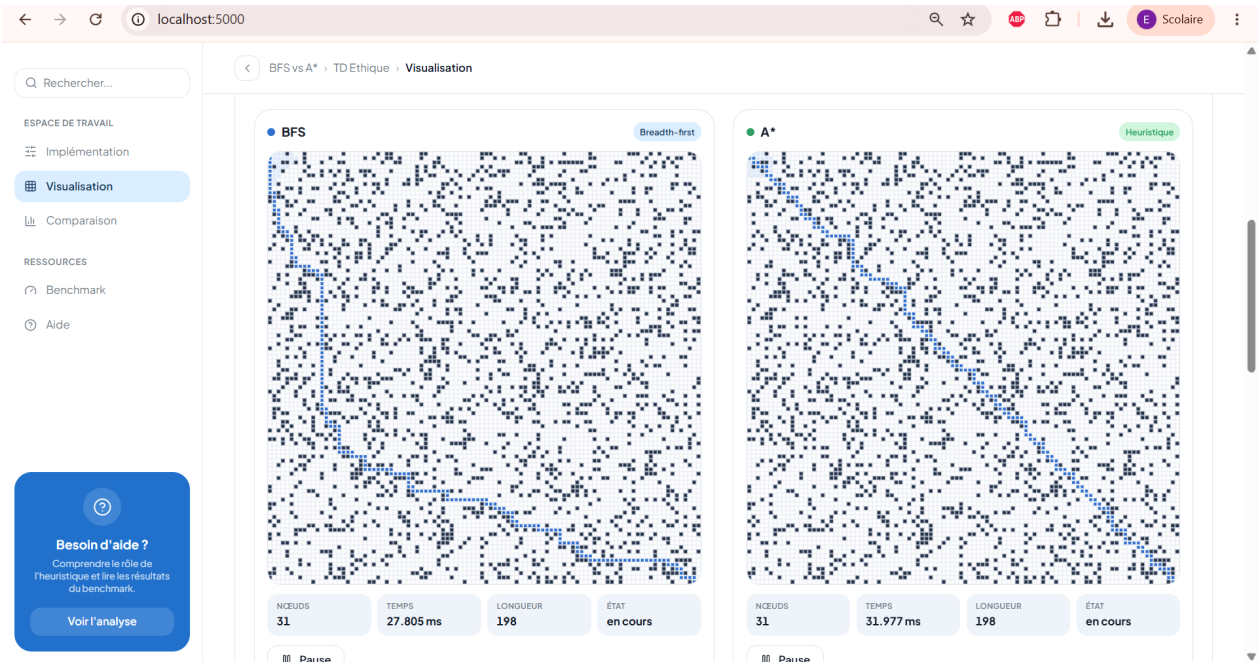


FIGURE 5 – Chemins trouvés sur une grille 100×100 : même longueur (198) pour BFS et A*.

3. Graphiques

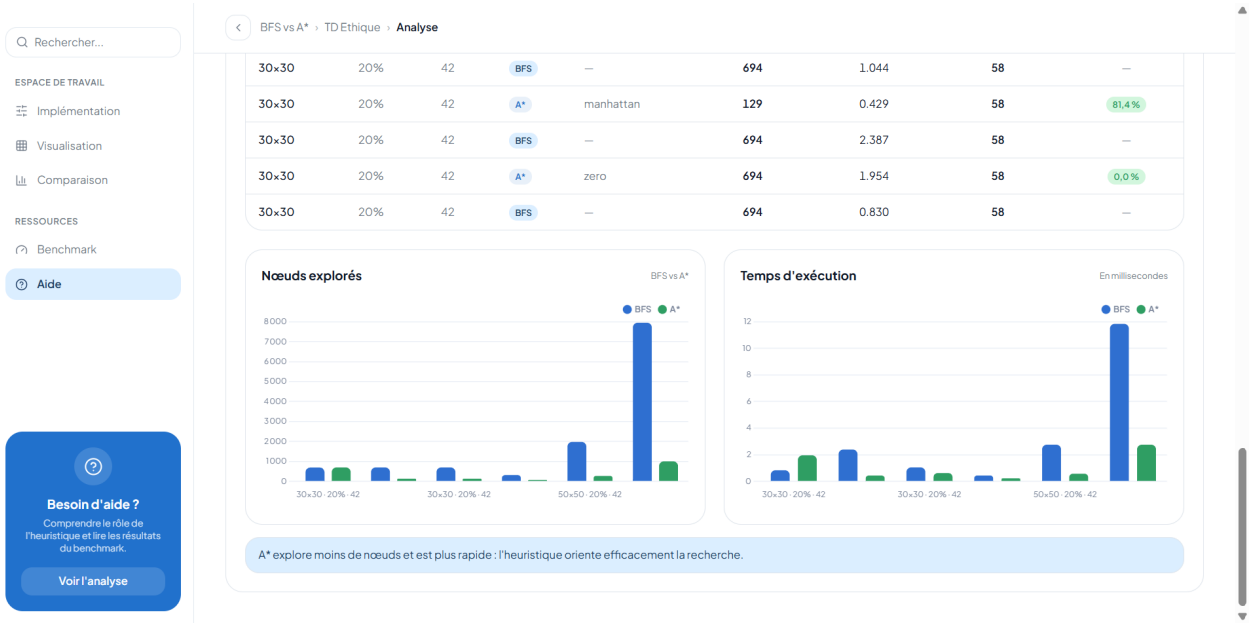


FIGURE 6 – Nœuds explorés et temps d'exécution, BFS (bleu) contre A* (vert), pour plusieurs tailles de grille.

L'écart en nombre de nœuds augmente avec la taille de la grille : BFS passe de 311 à 7944 nœuds, soit un facteur 25, tandis que A* passe de 65 à 999 nœuds, soit un facteur 15.

4. Influence du choix de l'heuristique

Sur la grille 100×100, avec l'heuristique de Chebyshev, A* explore 7384 nœuds contre 999 avec Manhattan et 7944 pour BFS, soit une réduction de seulement 7 % par rapport à BFS.

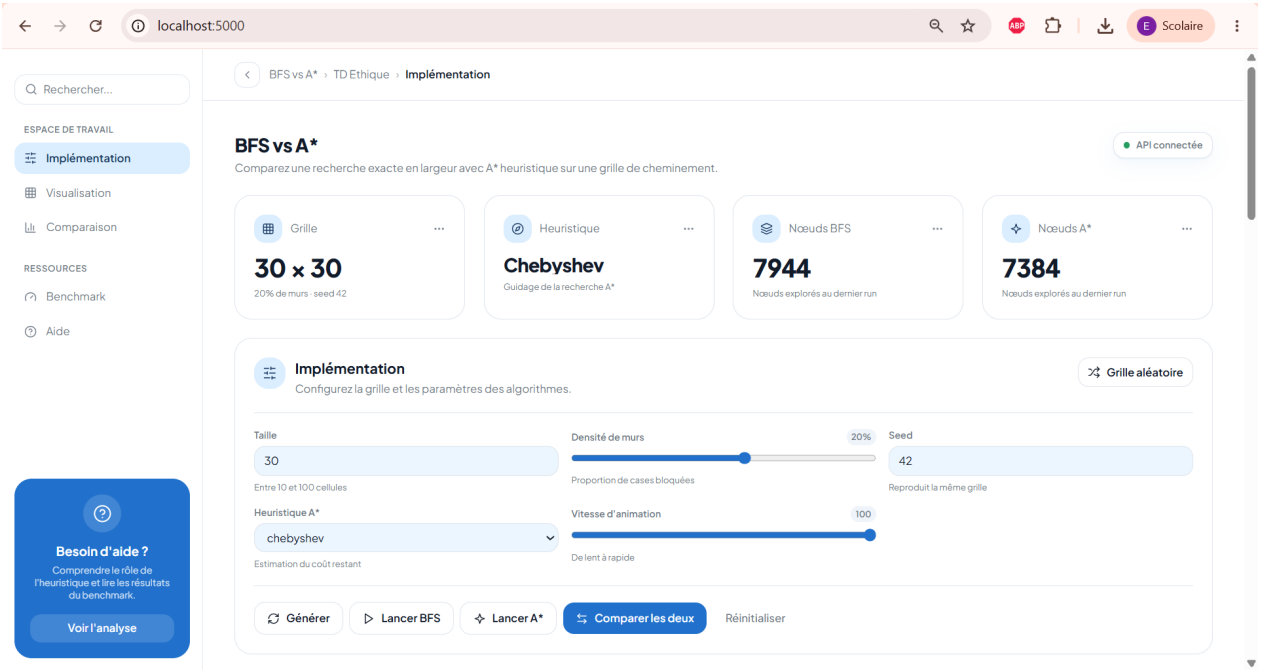


FIGURE 7 – Résultat avec l'heuristique de Chebyshev : 7944 nœuds pour BFS et 7384 pour A*.

Heuristique	**Nœuds explorés**	**Contexte**
Zéro (Dijkstra)	= BFS (694 sur 30×30)	aucune information
Chebyshev	7384 (100×100)	admissible mais peu informée
Manhattan	999 (100×100)	informée et admissible

TABLE 2 – Effet de la qualité de l'heuristique sur l'exploration.

Partie 3 : Analyse et réponses aux questions

1. Pourquoi A* est-il plus efficace ?

BFS explore sans préférence de direction : il développe tous les nœuds à distance 1, puis 2, puis 3, et ainsi de suite, ce qui forme une zone qui s'élargit dans toutes les directions autour du départ. A* classe les nœuds selon $f = g + h$ et développe d'abord ceux qui semblent les plus proches de l'arrivée. Il ignore donc une grande partie des cases éloignées du but.

Les mesures le confirment : A* explore entre 79 % et 87 % de nœuds en moins, et cet écart augmente avec la taille de la grille. Sur 100×100, il est environ 4 fois plus rapide (2,75 ms contre 11,83 ms) et trouve un chemin de même longueur, donc optimal. L'efficacité de A* ne coûte pas la qualité de la solution.

Les temps d'exécution restent plus bruités que les nœuds explorés. Sur 30×30, deux exécutions identiques de BFS ont donné 1,044 ms et 2,387 ms. De plus, A* utilise une file de priorité plus coûteuse que la simple file de BFS. Sur de petites grilles ou avec une mauvaise heuristique, le gain en temps est donc faible, voire négatif (1,954 ms contre 0,830 ms avec $h = 0$). Le nombre de nœuds explorés est un indicateur plus fiable, et les temps de l'application sont des médianes de plusieurs exécutions en mode benchmark.

2. Quel rôle joue la fonction heuristique ?

L'heuristique est la connaissance du problème donnée à l'algorithme : elle estime le coût restant et oriente la recherche. Son rôle dépend de deux propriétés :

- **Admissibilité** : $h(n)$ ne surestime jamais le coût réel. C'est ce qui garantit l'optimalité de A*. Manhattan est admissible sur une grille à quatre directions.
- **Précision** : plus h est proche du coût réel, moins A* explore de nœuds. Avec $h = 0$, A* ne reçoit aucune information et se comporte comme BFS (694 nœuds dans les deux cas). Chebyshev, qui sous-estime fortement le coût en l'absence de diagonales, guide peu (7384 nœuds). Manhattan, plus proche du coût réel, ne nécessite que 999 nœuds.

Une heuristique qui surestimerait le coût rendrait A* encore plus rapide, mais il pourrait alors renvoyer un chemin non optimal. Le choix de l'heuristique est donc un compromis entre rapidité et garantie de qualité.

3. Dans quels cas BFS reste-t-il préférable ?

- **Absence d'heuristique fiable** : si l'on ne dispose pas d'une bonne estimation du coût restant, A* n'apporte rien (cas $h = 0$).
- **Petits graphes** : le surcoût de la file de priorité peut annuler le gain. BFS est plus rapide dans nos mesures avec $h = 0$.
- **Simplicité et robustesse** : BFS est très simple à implémenter, sans paramètre à choisir ni risque d'heuristique mal conçue.
- **Objectifs multiples ou inconnus** : si l'on cherche tous les nœuds accessibles ou la distance vers plusieurs cibles, BFS donne ces distances en un seul parcours, alors que A* est conçu pour un but précis.
- **Garantie d'optimalité sans condition** : sur un graphe à coûts unitaires, BFS est optimal sans exiger d'hypothèse sur h.

Lien avec l'éthique de l'IA

Ce TD illustre un compromis qui revient dans tout système d'IA : l'efficacité obtenue grâce à une connaissance ajoutée (l'heuristique) dépend de la qualité de cette connaissance.

Une heuristique bien choisie accélère la décision sans en dégrader la qualité, alors qu'une heuristique inadaptée peut conduire à un résultat sous-optimal sans que l'utilisateur le remarque. Il est donc important de documenter les hypothèses d'un algorithme, de vérifier ses garanties (ici l'admissibilité) et de mesurer ses résultats plutôt que de se fier à sa réputation. Les deux algorithmes étudiés restent par ailleurs transparents et explicables : on peut visualiser chaque étape de leur raisonnement, ce qui est loin d'être le cas des modèles de type « boîte noire ».

Conclusion

Sur nos expériences, BFS et A* trouvent des chemins de même longueur, optimaux, mais A* avec l'heuristique de Manhattan explore 79 % à 87 % de nœuds en moins et gagne un facteur 4 en temps sur les grandes grilles. L'avantage de A* augmente avec la taille du problème et dépend de la qualité de l'heuristique : avec $h = 0$ il ne bat pas BFS, et avec une heuristique peu informée (Chebyshev sans diagonales) le gain est très limité. BFS reste le bon choix pour les petits problèmes, en l'absence d'heuristique fiable, ou lorsqu'on veut une solution simple et garantie.